

Open in app ↗



★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



Retrieval-Augmented Generation (RAG): The Complete Guide

11 min read · Just now



Kunal Khanvilkar



From the origins to agentic systems and beyond— what RAG is, why it exists, how it works, and which type solves which problem.

Why language models needed a memory fix

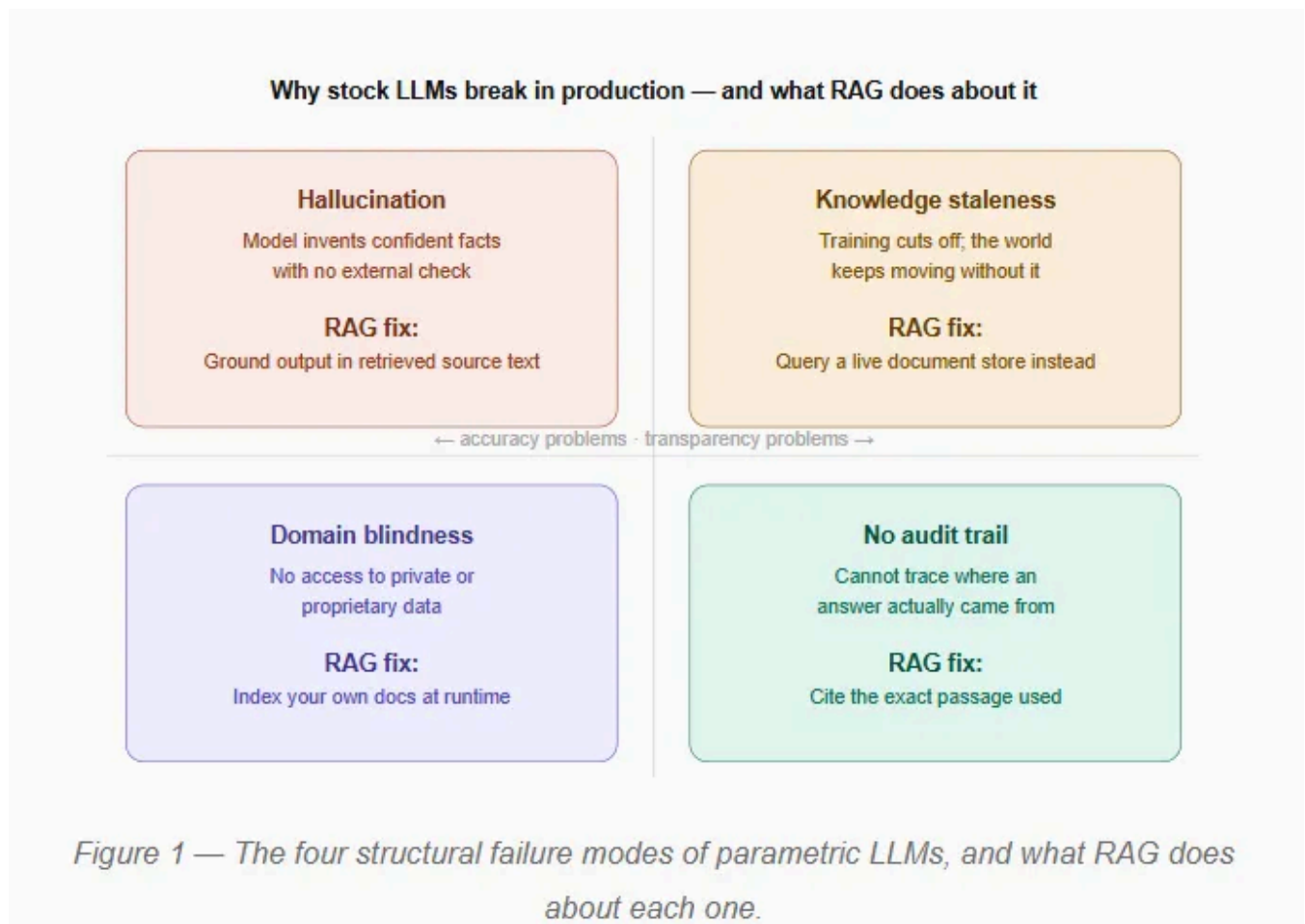
Every large language model ships with an invisible expiry date stamped into its weights. Once training ends, the clock stops — but the world continues. Ask a model about a policy update from last quarter, a competitor's product launched last month, or an internal document your team wrote last week, and you get one of two bad outcomes: a confidently wrong fabrication, or a polite admission of ignorance. Neither is tolerable when you are building something people actually rely on.

Retrieval-Augmented Generation was the research community's answer to this problem. The original paper, published in 2020 by Lewis and colleagues at Meta AI, proposed a conceptually simple but powerful fix: instead of expecting the model to remember everything it might ever need, give it the ability to look things up. Pair the generative engine with a retrieval mechanism, and suddenly the model does not need to carry all knowledge inside its weights — it just needs to know how to use what it finds.

That idea has since grown into an entire field. In 2024 alone, over 1,200 RAG-related papers appeared on arXiv. The architecture has become the default approach for deploying

language models in organizations working with proprietary, sensitive, or rapidly changing information.

The four failure modes RAG was designed to fix



These four failure modes are not bugs to be patched — they are structural properties of how parametric models store knowledge. Hallucination happens because generation is unconstrained by external reality. Staleness is baked into the training cutoff. Domain blindness exists because most useful organizational knowledge never appeared in a public training corpus. And auditability is impossible when the model cannot point to a source it used, because there was no source — only statistical memory.

RAG addresses each one directly: retrieved evidence keeps answers honest, a live document store replaces stale weights, your private corpus becomes the model's working memory, and cited sources make every claim traceable back to its origin.

The mechanics: what a RAG pipeline actually does

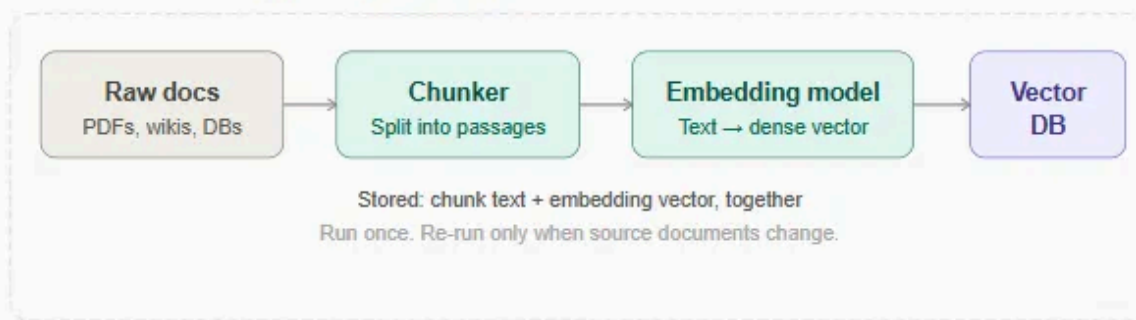
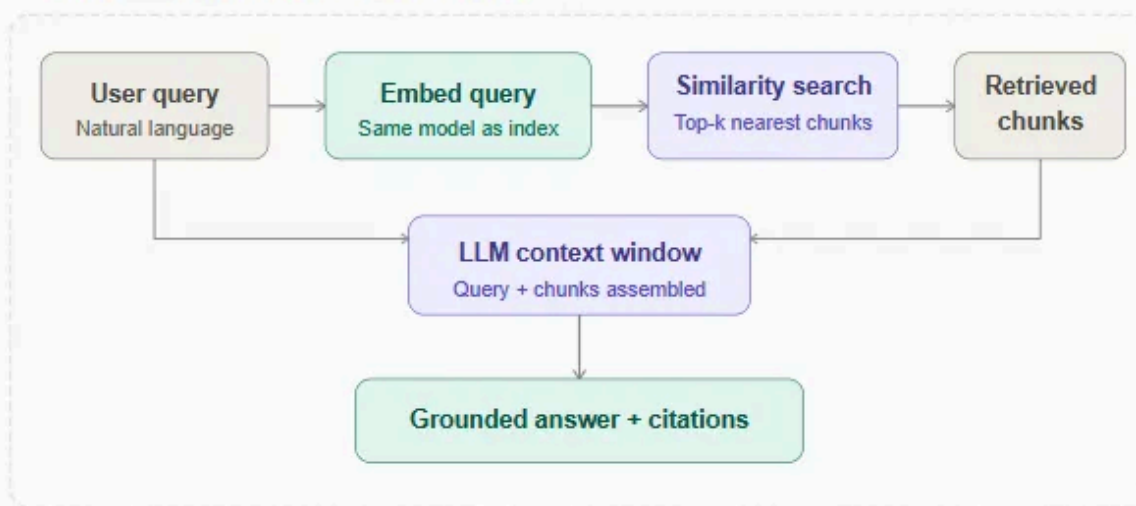
Phase 1 — Indexing (offline, runs once)**Phase 2 — Inference (online, every query)**

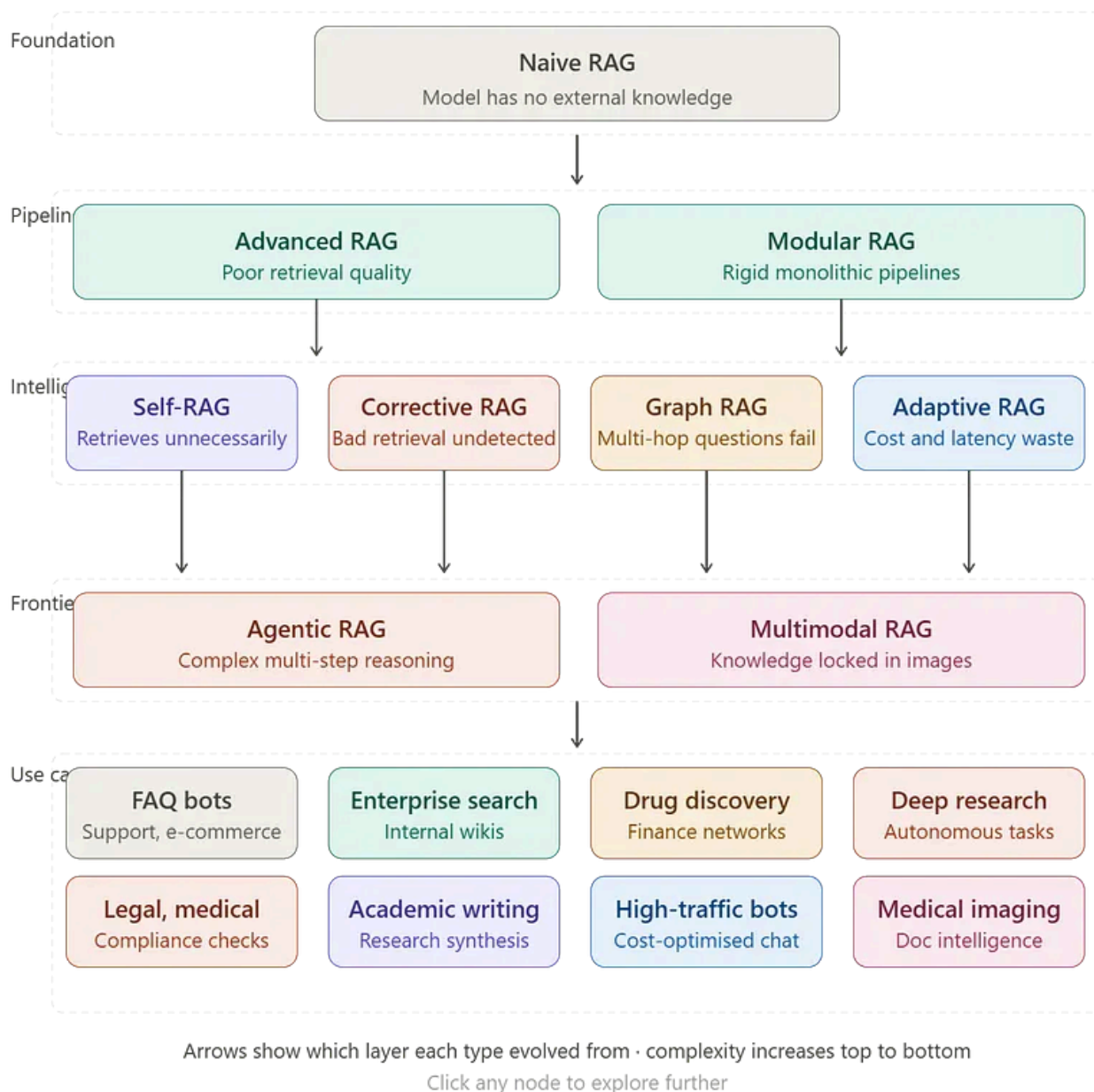
Figure 2 — The two-phase RAG pipeline. Indexing happens once offline; inference runs on every query.

A RAG system lives in two separate time zones. The indexing phase happens once, offline — your documents are broken into chunks, each chunk is converted into a vector by an embedding model, and both the vector and the original text land together in a vector database. This is the foundation. When a user later asks a question, that query is embedded using the exact same model (ensuring the math is comparable), and the database returns whichever stored chunks sit closest in vector space. Those chunks get loaded into the LLM's context window alongside the original question. The model reads both and writes an answer it cannot stray far from — because the evidence is right there in front of it.

The key insight: RAG separates what the model knows from what the model can access. You are not changing the model — you are changing its information environment at the moment of generation.

RAG Landscape

RAG type landscape — problem solved at each layer



Each layer in this diagram exists because the layer below it left a specific gap. Naive RAG gave models memory but not judgment. Advanced and Modular RAG improved the pipeline's mechanical quality without changing who made decisions. The intelligence layer — Self-RAG, Corrective RAG, Graph RAG, Adaptive RAG — pushed decision-making into the system itself. At the frontier, Agentic RAG collapses the entire pipeline into an open-ended reasoning loop, and Multimodal RAG extends the whole stack to data that was never text to begin with.

1. Naive RAG — the starting point

The original design: embed documents, retrieve the top-k most similar chunks, pass them to the LLM, generate. Fast to build and effective for straightforward use cases, but retrieval quality lives and dies by how well the raw query matches the indexed text. Poorly phrased questions retrieve poor context, and the model has no way to know whether what it received is trustworthy. Best for: FAQ bots, product support, internal knowledge bases where queries are clean and stakes are low.

2. Advanced RAG — engineering the retrieval step

Advanced RAG wraps additional intelligence around the core retrieve-generate loop. Before retrieval: query rewriting rephrases the question, HyDE generates a hypothetical answer and searches with that instead, and query decomposition breaks compound questions into simpler sub-questions. After retrieval: cross-encoder rerankers re-score each passage specifically for the query, and context compression trims irrelevant sentences. Best for: enterprise search over large unstructured document collections where users ask in natural, unoptimized language.

3. Modular RAG — pipelines as Lego

Modular RAG is less an algorithm and more an architectural philosophy. The pipeline is decomposed into independent, interchangeable components — retriever, reranker, reader, router, memory — each of which can be upgraded without touching the rest. Real production systems evolve: the embedding model you chose in 2023 may not be the best one available today. Modular design makes iteration possible without rebuilding from scratch. Best for: any team building a RAG system they intend to operate and improve over time.

4. Self-RAG — the model as its own critic

Introduced by Asai et al. in 2023, Self-RAG trains the model to decide for itself when retrieval is needed, and to critique the relevance and faithfulness of what it retrieves. Special reflection tokens act as internal checkpoints: should retrieval happen here? Are the passages genuinely relevant? Does the answer stay faithful to the evidence? The result is a model that skips retrieval when its own knowledge is sufficient, and actively challenges weak context. Best for: research assistance, academic writing tools, and knowledge synthesis tasks where the right move is sometimes retrieval and sometimes reasoning.

5. Corrective RAG (CRAG) — building in a safety net

CRAG, published by Chen et al. in early 2024, treats retrieval quality as something that must be verified, not assumed. After documents are retrieved, a lightweight evaluator scores each one for relevance. Strong scores: proceed. Borderline scores: rewrite the query and retry. Consistently poor scores: fall back to web search. In a naive pipeline, bad retrieval produces bad answers with no indication anything went wrong. CRAG breaks that pattern. Best for: legal research, clinical information systems, regulatory compliance — anywhere a confidently wrong answer carries real consequences. (arXiv:2401.15884)

6. Graph RAG — when relationships matter more than similarity

Vector search is fundamentally a similarity operation — it finds passages that look like the query. That works well for direct lookups but fails for questions requiring connections across multiple documents: chains of events, corporate relationships, biological pathways, legal precedents. Graph RAG, developed at Microsoft in 2024, layers a knowledge graph on top of the vector index. Entities and their relationships are extracted and stored as graph nodes and edges. Multi-hop questions that vector similarity cannot navigate are handled through graph traversal. Best for: drug discovery, financial intelligence, legal analysis involving case law networks.

7. Adaptive RAG — routing by complexity

Adaptive RAG, from KAIST researchers in 2024, attacks a different problem: waste. A learned router classifies each query before anything else triggers. Simple factual questions the model already knows skip retrieval entirely. Medium-complexity questions get one pass. Genuinely complex multi-hop questions trigger iterative retrieval rounds. At the scale of millions of daily queries, reducing unnecessary retrieval saves significant compute and also reduces retrieval noise — fewer irrelevant passages contaminating responses to questions the model already had the answer to. (arXiv:2403.14403)

8. Agentic RAG — retrieval as a reasoning loop

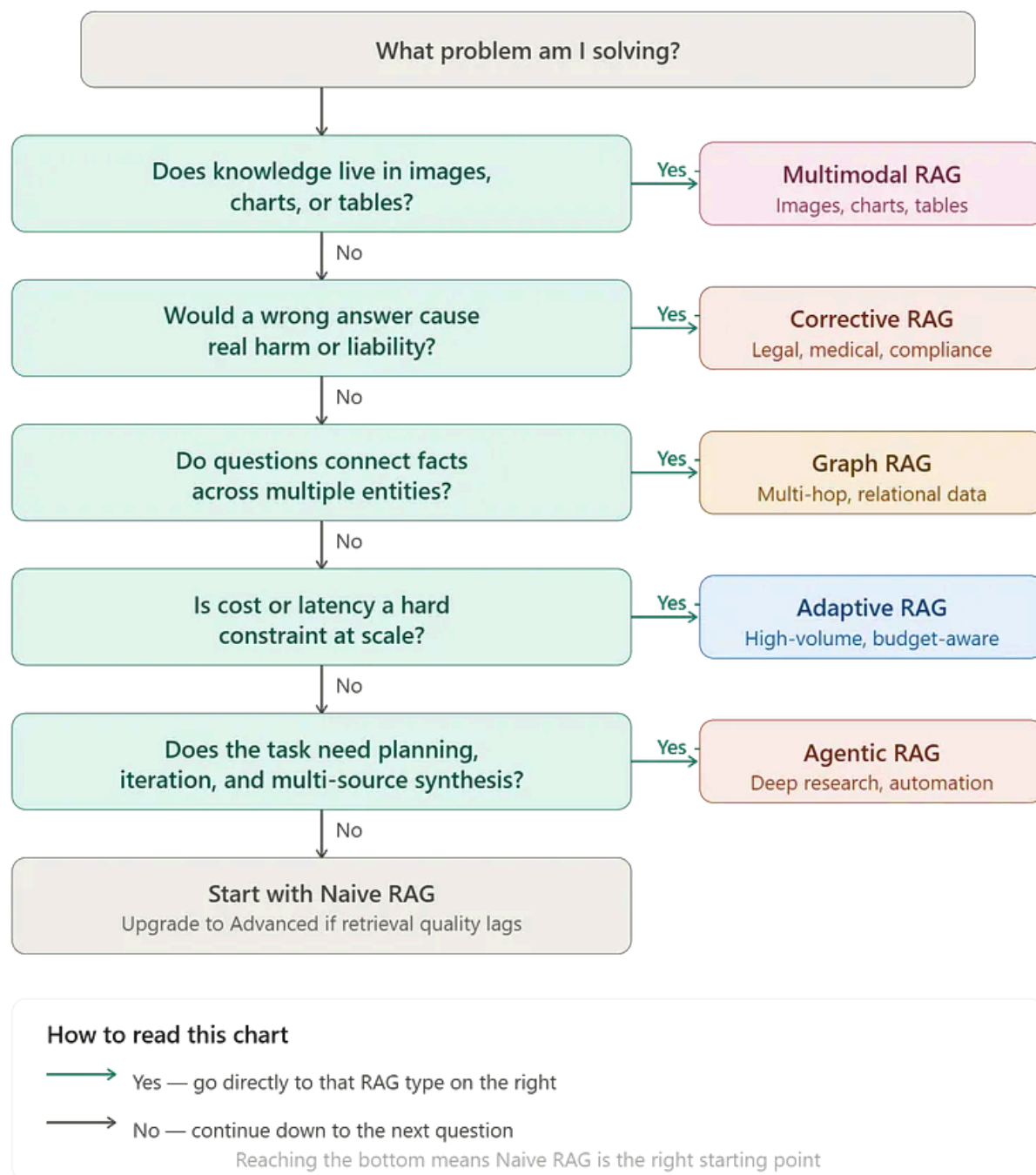
All previous RAG types follow a predetermined script: retrieve once (or a fixed number of times), then generate. Agentic RAG abandons the script. An LLM agent controls the retrieval process entirely: decide what it needs to know, choose which tool to invoke (vector search, web search, SQL query, API call), evaluate the result, decide whether to continue searching, and iterate until it is satisfied. A complex question might trigger five iterations across three different sources. By 2025, this evolved into Reasoning Agentic RAG, where decision-making is embedded inside

the model's own reasoning trajectory rather than orchestrated externally. Best for: deep research tasks, competitive intelligence, autonomous document workflows.

Multimodal RAG — beyond text

Multimodal RAG extends the index beyond text to images, tables, charts, diagrams, and audio. Vision models process visual content, OCR handles structured tables, and the retrieval pipeline can return image patches or data extracts alongside text. For domains where knowledge lives in formats other than prose, this is not optional — it is essential. Best for: medical imaging, engineering documentation, financial reports heavy with charts and tables.

How to pick the right type



The decision tree above is a triage tool, not a checklist. Start by describing the primary failure you need to prevent, not the architecture you want to build. If no branch triggers before reaching the bottom, that is not a failure of the framework — it is a signal that Naive RAG is genuinely the right starting point, and any additional complexity would be solving a problem you do not yet have.

The problem you need to solve	The architecture to reach for
Model lacks access to your documents	Naive RAG
Users ask messy, underspecified questions	Advanced RAG
The system needs to evolve without a rebuild	Modular RAG
Model retrieves when it does not need to	Self-RAG
Wrong answers carry real-world consequences	Corrective RAG
Questions require connecting facts across sources	Graph RAG
Retrieval cost and latency need managing at scale	Adaptive RAG
Tasks require multi-step planning and synthesis	Agentic RAG
Knowledge lives in images, tables, or charts	Multimodal RAG

The Evolution in Brief

How RAG evolved: 2020 to 2025

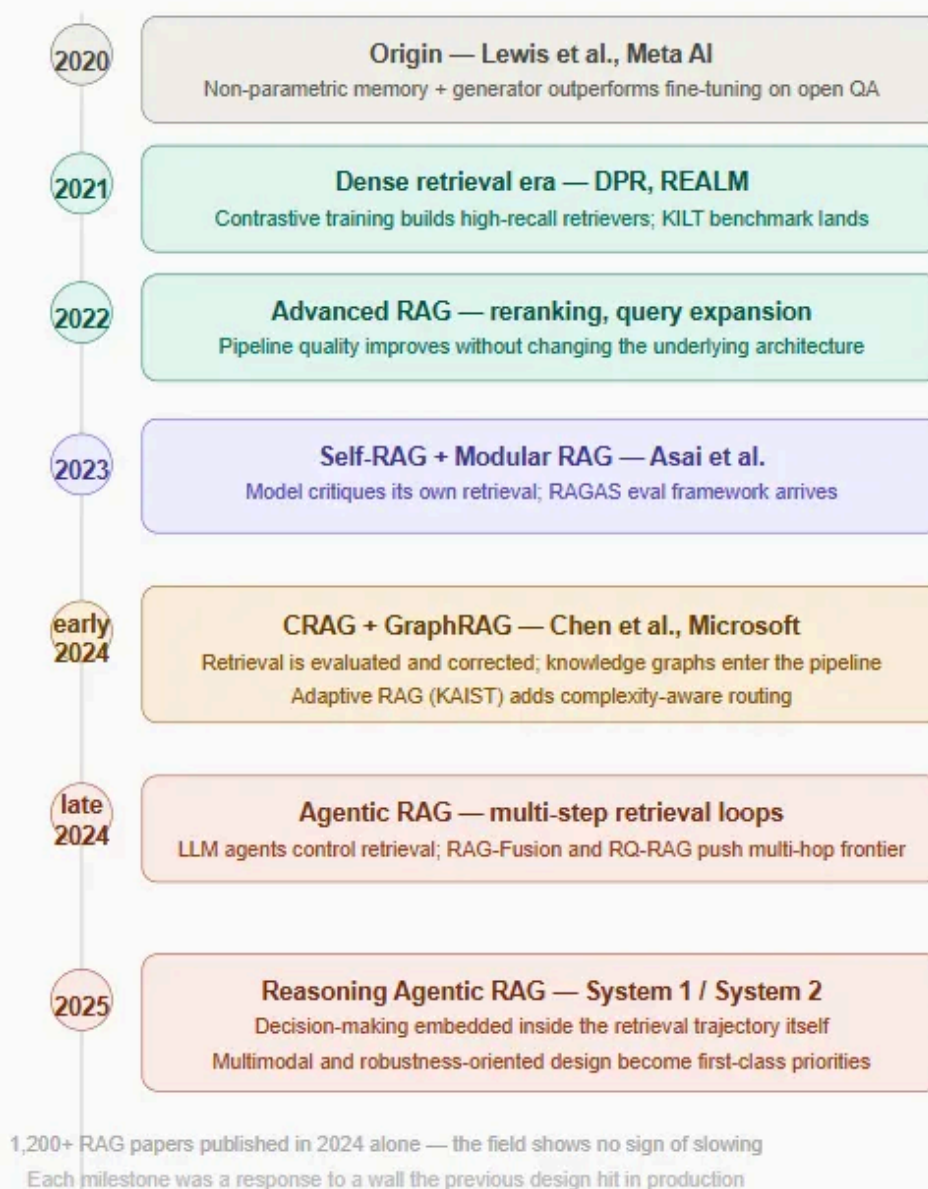


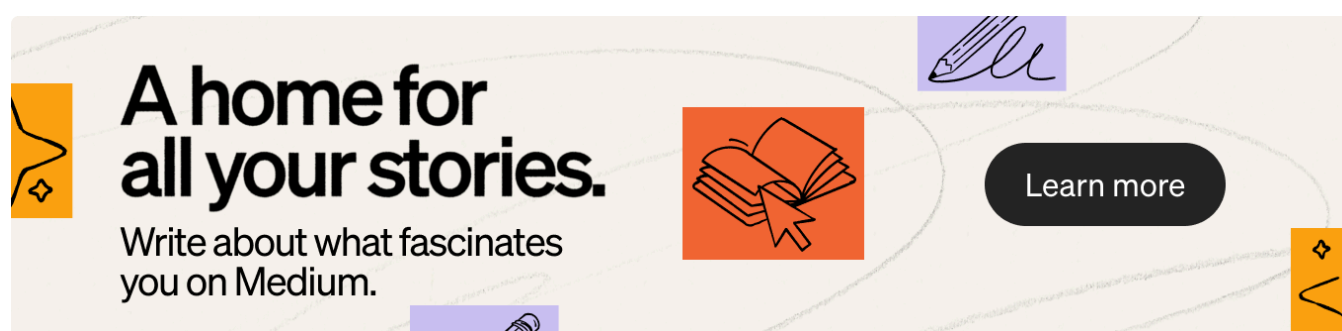
Figure 5 — The RAG evolution timeline from the 2020 origin paper to Reasoning Agentic RAG in 2025.

What the timeline makes visible is that RAG's evolution was not a planned roadmap — it was a series of responses to walls practitioners kept hitting. The 2020 paper proved the concept. Dense retrieval in 2021 made it fast enough to matter at scale. Advanced RAG in 2022 addressed the quality gap that emerged once the concept was stress-tested on real corpora. Self-RAG in 2023 shifted the question from “how do we retrieve better?” to “when should we retrieve at all?” — a conceptually different problem that required a different kind of answer. 2024 delivered three parallel advances almost simultaneously: a way to verify and correct bad retrieval, a way to handle relational knowledge that vector similarity cannot capture, and a way to

route by complexity to contain cost. By late 2024, the agent had taken over the loop entirely. By 2025, even that felt too rigid — the new frontier is systems where what to retrieve and when to stop is reasoned through from inside, not orchestrated from outside.

What actually matters when you build it

Chunking decisions are underestimated. The way you split documents profoundly affects what gets retrieved. Chunks too small strip away the context that makes a passage interpretable. Chunks too large dilute the relevance signal and consume precious context window space. The right size depends on your document type, query patterns, and model context length. Test empirically before committing to a strategy.



Hybrid retrieval is almost always worth it. Sparse retrieval (BM25, keyword matching) handles exact term matches and rare proper nouns that dense vectors tend to miss. Dense retrieval handles conceptual matches where the wording differs entirely. They fail in complementary ways, which is precisely why combining them consistently outperforms either approach alone.

You cannot improve what you do not measure. The RAGAS framework gives you faithfulness (does the answer stay within what was retrieved?), answer relevance (does it actually address the question?), and context precision (was the retrieved context actually needed?) — all without requiring human-labeled ground truth. If these numbers are not being tracked, you are operating blind.

Retrieval failures are invisible without observability. When a RAG system gives a wrong answer, the cause is almost always in what was or was not retrieved, not in the generation step. You cannot diagnose this without logging the retrieved context alongside the response. Treating generation and retrieval as a single black box makes debugging nearly impossible.

Sophistication is not the same as reliability. An Agentic RAG system in a notebook that has never been stress-tested is not superior to a well-tuned Naive RAG system serving real users and being actively monitored. Build the simplest architecture that addresses your actual failure mode, measure it carefully, and add complexity only when evidence demands it.

New research frontiers (2026 and upwards)

- **Model-driven retrieval agency (A-RAG, Feb 2026):** Rather than following a fixed retrieval script, A-RAG hands hierarchical retrieval tools — keyword search, semantic search, and chunk-level reading — directly to the model, letting it decide what to look up, at what granularity, and whether to keep searching. The system scales as the underlying model improves, because retrieval decisions are now the model's problem to solve, not the engineer's procedure to prescribe. (*Du et al., arXiv:2602.03442*)
- **Agentic RAG taxonomy and safety formalization (SoK, Mar 2026):** As Agentic RAG systems have grown in complexity, the field has started demanding structure. A comprehensive Systemization of Knowledge paper from March 2026 builds a formal taxonomy categorizing agentic systems by their planning mechanisms, retrieval orchestration strategies, memory representations, and coordination patterns — laying the groundwork for consistent evaluation benchmarks and governance frameworks that the field has lacked until now. (*Mishra et al., arXiv:2603.07379*)
- **Persistent agent memory as a cognitive resource:** Early 2026 saw a burst of work reconceiving memory in RAG systems — not as a passive vector index queried at inference time, but as a structured, writable, self-organizing resource that agents can actively manage across sessions. Systems like MAGMA (Multi-Graph based Agentic Memory Architecture) and EverMemOS treat memory as something the agent maintains and reorganizes, enabling coherent long-horizon reasoning that single-session retrieval cannot support.
- **Multi-agent RAG coordination:** Research like O-Researcher and MA-RAG has moved beyond single-agent retrieval loops toward collaborative architectures where multiple specialized agents divide retrieval labor, cross-check each other's findings, and synthesize answers from independently gathered evidence. This is particularly effective for deep research tasks where a single agent's context window becomes the bottleneck before reasoning quality does.

- **Privacy-preserving RAG for regulated industries:** As RAG systems move deeper into healthcare, legal, and financial workflows, the security surface they expose has become a research priority in its own right. Demonstrated adversarial extraction attacks — where carefully crafted queries cause the system to leak private knowledge base contents — have pushed the field toward differential privacy techniques, federated RAG architectures that keep sensitive data local, and retrieval-time access controls that enforce document-level permissions without exposing what was or wasn't retrieved.
- **Robustness-oriented retrieval design:** A recognized gap in current RAG systems is their fragility under distribution shift, adversarial inputs, and noisy corpora. New self-supervised approaches like SEER and RAG-Ex use internal feedback signals — pseudo-relevance labels and generation consistency comparisons respectively — to filter out unreliable retrieved passages without requiring human-annotated ground truth, reducing context noise while improving faithfulness to source material.
- **LLM-agnostic and cloud-native RAG platforms:** A practical shift underway in 2026 is the decoupling of RAG infrastructure from any specific model. Production-grade RAG platforms are being designed to swap the underlying LLM without rebuilding the retrieval pipeline, giving organizations the flexibility to move between models as costs, capabilities, and compliance requirements shift — treating the retrieval layer as stable organizational infrastructure rather than something rebuilt whenever the model changes.

References

Lewis et al. (2020) — Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks, Meta AI

Asai et al. (2023) — Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection

Chen et al. (2024) — Corrective Retrieval Augmented Generation, arXiv:2401.15884

Jeong et al. (2024) — Adaptive-RAG, KAIST, arXiv:2403.14403

Edge et al. (2024) — GraphRAG, Microsoft Research

Gupta et al. (2024) — A Comprehensive Survey of RAG, arXiv:2410.12837

Sharma (2025) — RAG: Architectures, Enhancements, and Robustness Frontiers, arXiv:2506.00054

Retrieval Augmented Gen

LLM

Agentic Rag

Rag Vs Agentic Rag

Adaptive Rag



Edit profile

Written by Kunal Khanvilkar

0 followers · 1 following

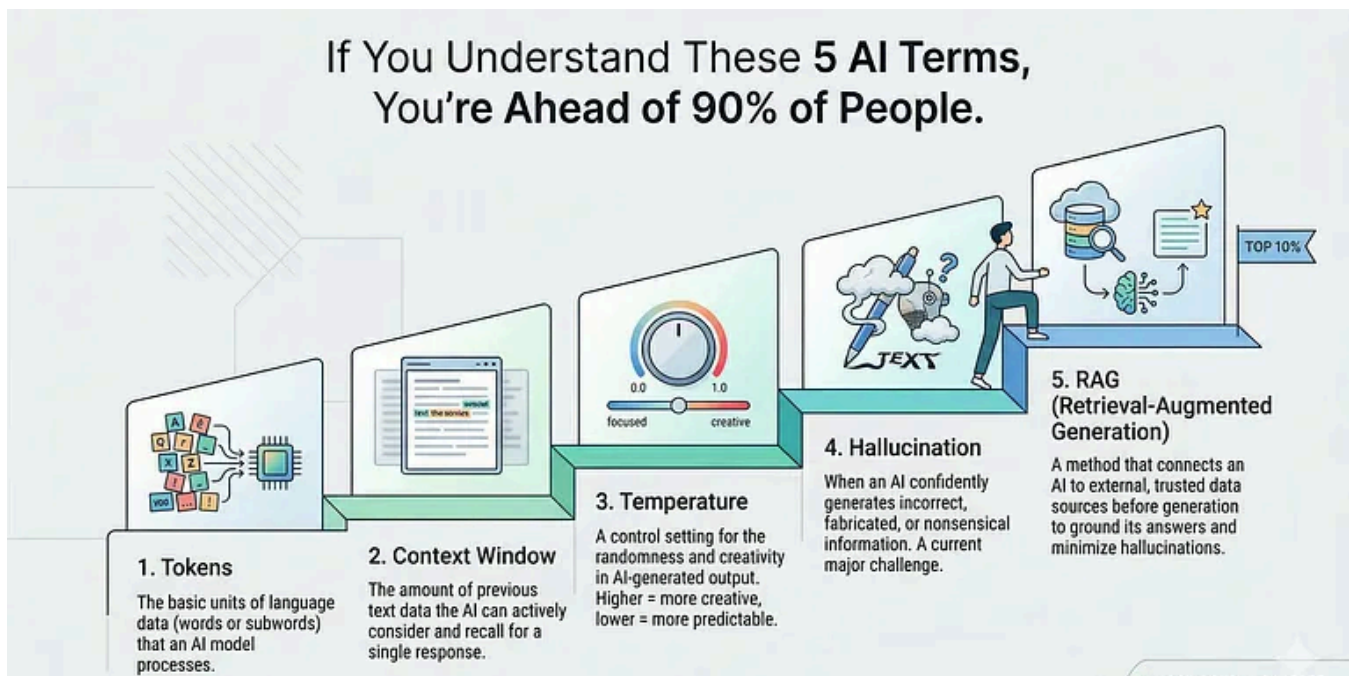
No responses yet



Kunal Khanvilkar

What are your thoughts?

Recommended from Medium



In Towards AI by Shreyas Naphad

If You Understand These 5 AI Terms, You're Ahead of 90% of People

Master the core ideas behind AI without getting lost



Mar 29

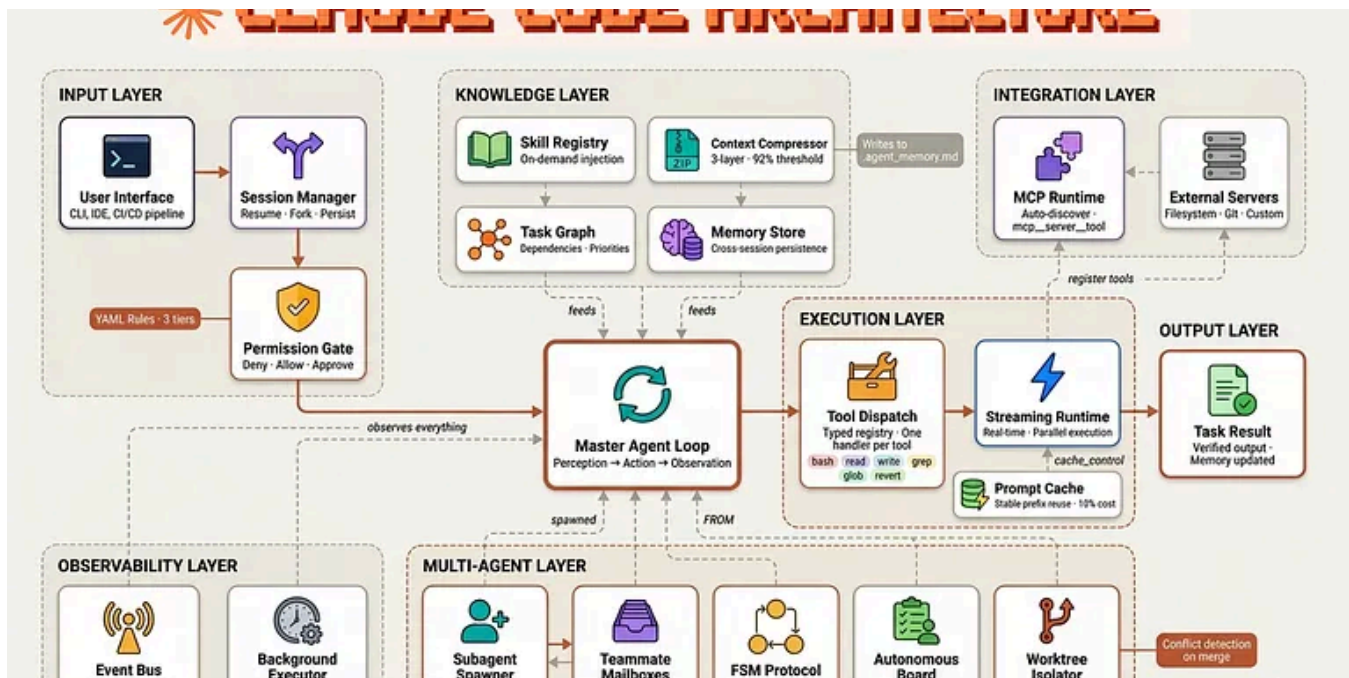


8.9K



180





In Level Up Coding by Fareed Khan

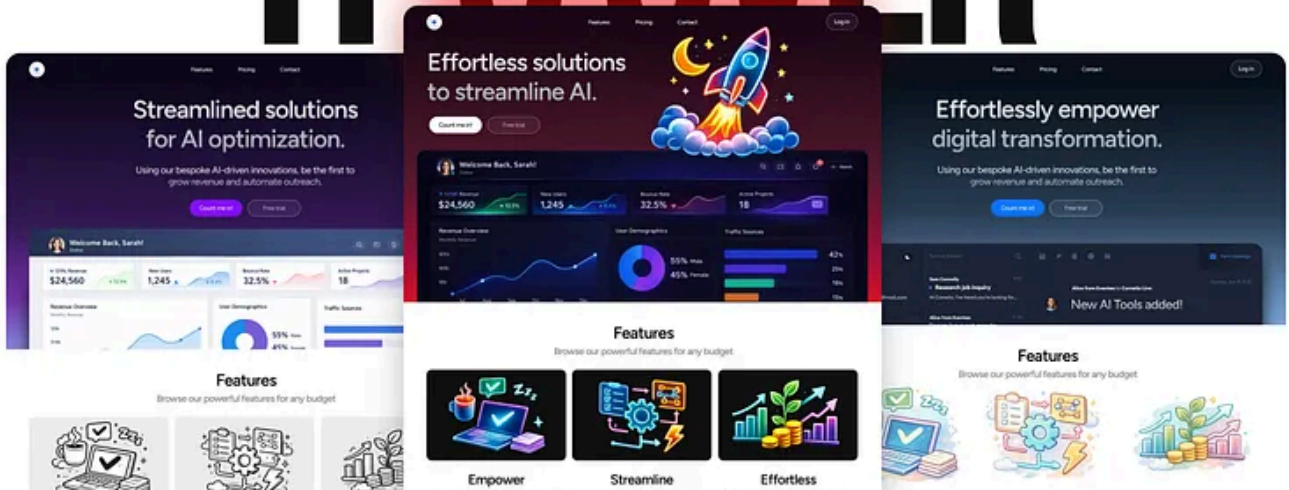
Building Claude Code with Harness Engineering

Multi-agents, MCP, skills system, context pipelines and more

★ 6d ago 🤝 1K 💬 8



IT'S OVER

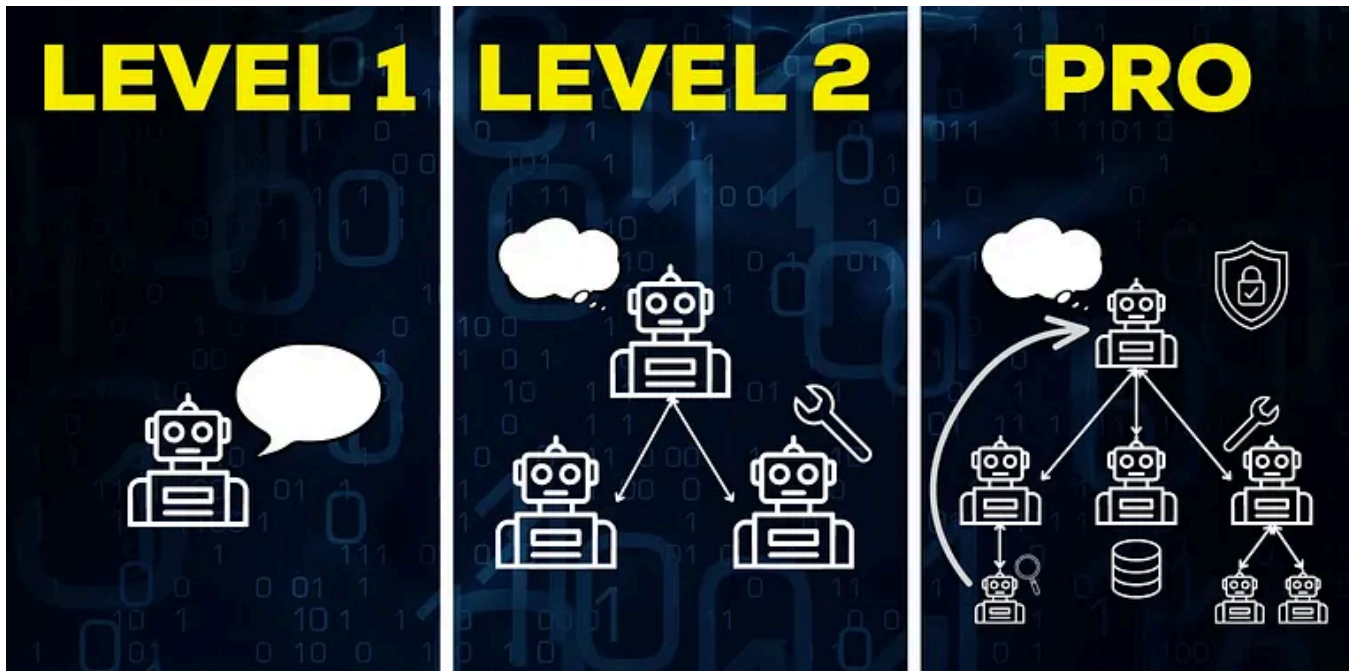


 Michal Malewicz 

Vibe Coding is OVER.

Here's What Comes Next.

🌟 Mar 24 🖱️ 5K 💬 188



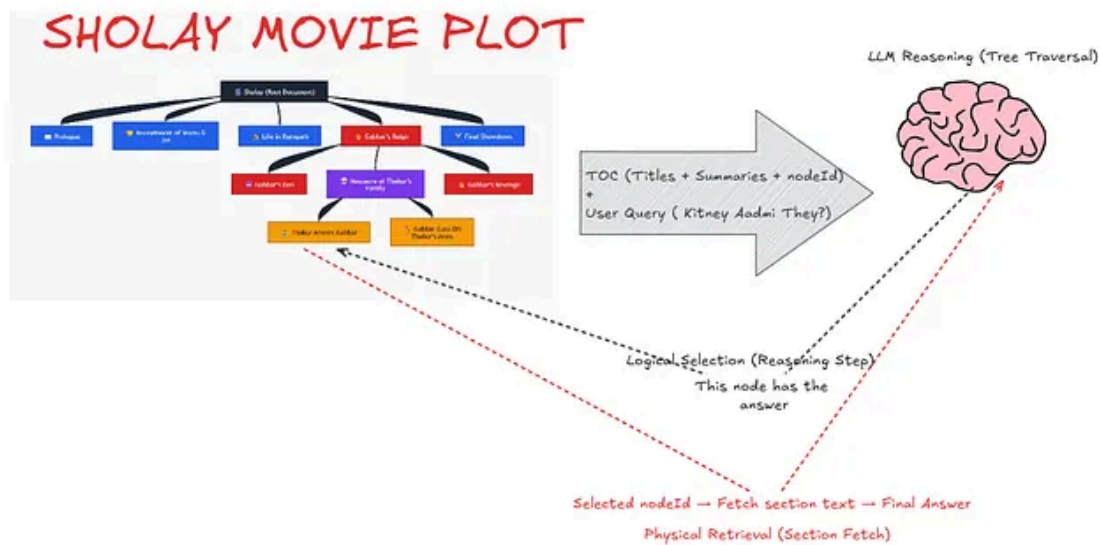
 In Data Science Collective by Marina Wyss

AI Agents: Complete Course

From beginner to intermediate to production.

🌟 Dec 6, 2025 🖱️ 5.7K 💬 229



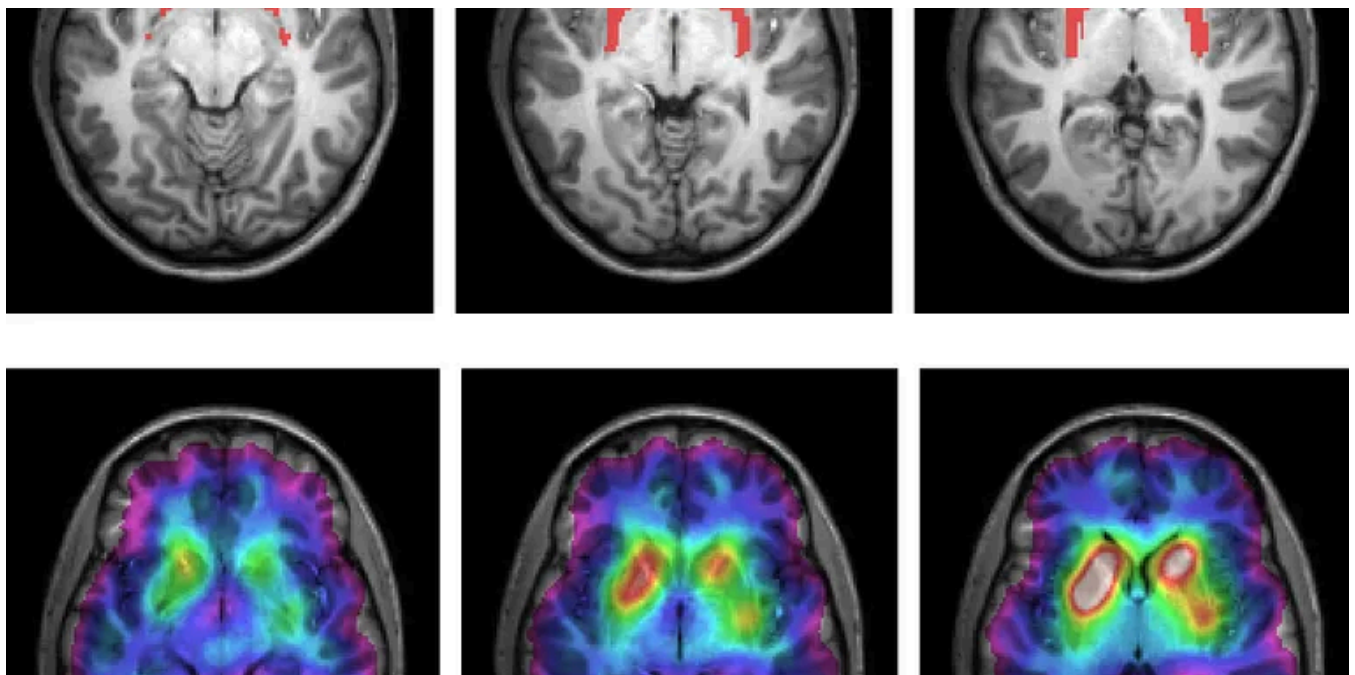


 Vishal Mysore

What Is PageIndex? How to Build a Vectorless RAG System (No Embeddings, No Vector DB)

PageIndex is a vectorless, reasoning-based Retrieval-Augmented Generation (RAG) approach that retrieves answers from long documents without...

Mar 1  210  11



In Write A Catalyst by Dr. Patricia Schmidt

As a Neuroscientist, I Quit These 5 Morning Habits That Destroy Your Brain

Most people do #1 within 10 minutes of waking (and it sabotages your entire day)

★ Jan 14 🖱 46K 💬 941



See more recommendations